A Novel Application of Ant Colony Optimization to k-defective Maximum Edge Bicliques

Kiyaan Pillai

July 2026

We apply Ant Colony Optimization to the maximum k-defective edge biclique problem, an NP-hard optimization. We compare its performance to other proposed heuristics and find it complements previously proposed heuristics to serve as an upper bound for . We find that our algorithm offers a promising ability to identify k-MDB with lower dependence on a tuned hyperparameter, θ, than previous approaches. We evaluate our approach on several small and large bipartite graphs, and identify several opportunities to further increase performance.

1 Introduction

A bipartite graph is a common graph structure where one set of vertices is connected solely to another. Bipartite graphs can model user-product behavior, person-place interaction, and more. A biclique is a set of vertices on a bipartite graph such that each vertex on one side is connected to each vertex on the other. The maximum edge biclique problem aims to identify the biclique with the most edges on a bipartite graph. The problem has applications in online recommendation systems, fraud detection, and community detection [1].

Real-world data often introduces noise to graphs, and structures with relatively few missing connections may still be relevant. A biclique that is allowed up to k missing edges, or k-defective biclique, can overcome this real-world noise. Identifying the maximum k-defective edge biclique (k-MDB) remains an emerging field.

The k-MDB problem is NP-hard. Unless P = NP, there is no algorithm guaranteed to find the maximum solution that can run in polynomial time. When the absolute optimum is not necessary, approximation techniques are useful to significantly reduce computation while providing meaningfully large bicliques.

Dorigo et al. first proposed Ant Colony Optimization (ACO) as a metaheuristic to solve difficult combinatorial optimization problems, specifically the Traveling Salesman Problem [2]. Several improvements have been proposed to the algorithm since its inception, most notably the Min-Max Ant System

(MMAS) [4]. We adapt MMAS as a heuristic approach to identifying the k-MDB. We observe it performs significantly faster than brute-force algorithms while still returning comparably large k-defective bicliques.

2 Background

2.1 Definitions

Define a bipartite graph as a graph $G = (U, V, E)$, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges between U and V. Define $n = |U| + |V|$ and $m = |E|$. Let $S = (U_S, V_S)$ denote a subset of vertices where $U_S \subseteq U$ and $V_S \subseteq V$. Let $G(S) = (U_S, V_S, E_S)$ be the induced subgraph of S where $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$. Define $\bar{E}(S) = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \notin E\}$ be the set of non-edges in a subgraph. Let a k-defective biclique be a subgraph S where $|\bar{E}(S)| \leq k$.

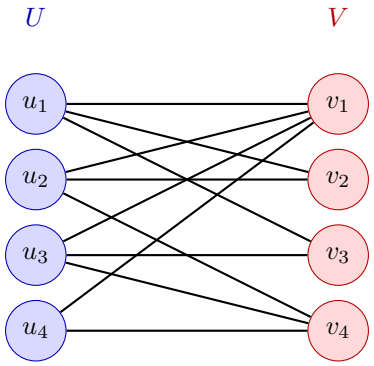

Figure 1: Example bipartite graph $G = (U, V, E)$

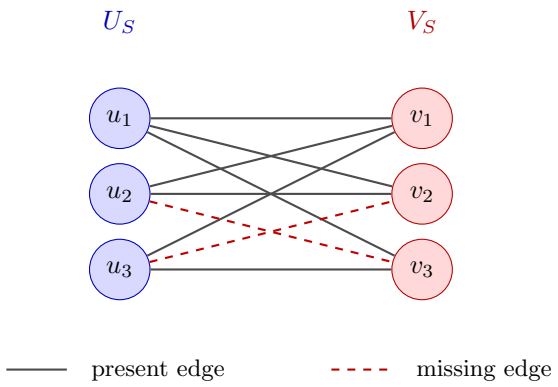

Figure 2: A 2-defective biclique present in Figure 1. Every vertex in U_S is connected to every vertex in V_S except for two missing edges. Since $|\bar{E}(S)| = 2$, this is a k-defective biclique, where $k \geq 2$.

Define the neighbors of a node u in a subgraph S as $N_S(u) = \{v \in S \mid \{u, v\} \in E(S)\}$. Define the nonneighbors of a node as $\bar{N}_S(u) = \{v \in S \mid (u, v) \notin E(S)\}$. Define the degree of a node as $d_S(u) = |N_S(u)|$, and the nondegree as $\bar{d}_S(u) = |\bar{N}_S(u)|$.

Define a maximal k-MDB as a subgraph S that cannot be expanded. That is, for any node $u \notin S$, $|\bar{E}(S \cup \{u\})| > k$. Define the maximum k-MDB as the largest maximal k-MDB in a graph G. Let θ be a user-defined limitation on the solution space. The algorithms proposed in this paper seek to find the maximum k-MDB where both sides have at least θ vertices. This is because, on real-world graphs, k-defective bicliques can be formed with many edges but disproportionate vertex counts, where one side has very few vertices while the other has many [1]. Without the θ term, the utility of the algorithm is limited.

2.2 Computational Complexity

Finding the maximum k-MDB on a bipartite graph is an NP-hard optimization problem. The problem reduces to the well-known NP-hard maximum clique problem, which seeks to find the maximum set of vertices on a graph connected to every other vertex [1].

2.3 Previous Work

Cui et al. [1] proposed a brute-force approach to the k-MDB problem, as well as an initial heuristic designed to find a k-MDB with exactly θ nodes on one side of the graph.

Dorigo et al. [2] proposed a metaheuristic for exploring graphs known as Ant Colony Optimization (ACO). Initially used on identifying the shortest path in problems such as the Traveling Salesman Problem, ACO has been adapted to several .

3 Methods

ACO traditionally has a set of ants that traverse a graph, leaving pheromone that attracts other ants to follow their path. As the ants explore the solution space, they all converge on one particular path as the quantity pheromones on that path becomes increasingly larger.

We adapt the ACO algorithm to the k-MDB problem. A population of ants iteratively adds vertices from G to a subgraph S, preferring nodes with higher d_S and higher pheromone τ.

Algorithm 1 ACO

Ensure: a subset of vertices S such that $G(S)$ is a k-MDB
  function ACO(G, N_e, N_g, E, T)
    for all $u \in G$ **do**
      $\tau(u) \leftarrow 1$
    $e \leftarrow 0$
    $B \leftarrow \{\}$
    while $e < N_e$ **do**
      $e \leftarrow e + 1$
      $A \leftarrow N_a$ empty subgraphs
      $\tau_C \leftarrow \tau$
      parallel for each $S_i \in S$ **do**
        while $|\bar{E}(S_i)| < k$ **do**
          $next \leftarrow$ softmax nodes in CANDIDATE(S_i) based on DESIRABILITY(u, S_i, τ_C)
          add $next$ to S_i
          $\tau_C(u) \leftarrow \tau_C(u) + T$
      $S_B \leftarrow \text{argmax}_{S_i \in S}$INSTANCEFITNESS($S_i$)
      if INSTANCEFITNESS(S_B) > INSTANCEFITNESS(B) **then**
        $B \leftarrow S_B$
      $\tau \leftarrow \tau_C$
      for all $u \in G$ **do**
        $\tau(u) \leftarrow \tau(u) \times E$
    return B
  function CANDIDATE(S)
    return $\{u \in G \mid k \geq |\bar{E}(S \cup \{u\})|\}$
  function DESIRABILITY(u, S, τ)
    $\eta \leftarrow d_S(u) + d_G(u) \times \sigma(|U_S| + |V_S|)$
    return $\tau(u) \times \eta^3$

We also propose several additions to this algorithm that empirically improve performance. We introduce an elitist-style selection process that deposits pheromones on the best solutions, as proposed by [Schiff]. To increase ant diversity, we softmax the solutions ranked by their fitness. We define the fitness function below.

Algorithm 2 Instance Fitness

function InstanceFitness($S = (U_S, V_S)$)
    $m \leftarrow 1$
    if $\min(|U_S|, |V_S|) \geq \theta$ **then**
        $m \leftarrow \max |U_S, V_S|$
    return $m \times \min(|U_S|, |V_S|)^2$

Additionally, to encourage bicliques with at least θ vertices on both sides, we limit ants to exploring vertices on the side with fewer in their current subgraph if at least one side does not have θ vertices.

Finally, we implement the Min-Max ACO system.

Algorithm 3 Repair

function Repair(G, S)
    $S_T \leftarrow$ tabu repair of S
    $S_G \leftarrow S$
    while $|\bar{E}(S_G)| < k$ **do**
        $C \leftarrow \{u \in G \setminus S_G \mid k \geq |\bar{E}(S_G \cup \{u\})|\}$
        if $|C| = 0$ **then**
            break
        append $\text{argmax}_{u \in G \setminus S_G} d_{S_G} u$
    return $\text{argmax}_{\{S \in S_T, S_G\}}$InstanceFitness($S$)

Finally, we implement the approach proposed by [Cui et al.]. [Cui et al.] observe that many k-MDB have one side with exactly θ vertices and propose a heuristic that removes nodes from one side while adding nodes to another until there are θ nodes on that side. We summarize their algorithm below.

Algorithm 4 Cui Heuristic

Ensure: a k-defective biclique of G
function CuiHeuristic(G)
    $U \leftarrow \{\}$
    $V \leftarrow V_G$
    while $|U| < \theta$ **do**
        append $u \in U_G \setminus U$ with largest d_V to U
        while $|\bar{E}(U \cup V)| > k$ **do**
            remove $v \in V$ with largest $\bar{d}_U$ from V
    return $U \cup V$

They then propose the following branch-and-pivot algorithm that takes advantage of this initial heuristic upper bound. The algorithm checks combinations of $S = (U_S, V_S)$ by maintaining a candidate set $C = (U_C, V_C)$ of all remaining nodes $u \in G$ s.t. $|\bar{E}(S \cup \{u\})| \leq k$ that could still be added to S.

It recursively branches on ($S' = S \cup \{u \in C\}, C' =$ candidate set of S') and ($S' = S, C' = C \setminus \{u \in C\}$). Several optimizations are added to make this algorithm faster than a naive brute-force approach. Its time complexity is $O(m\beta_k^n)$, where β is the largest real root of the equation $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$.

Algorithm 5 Branch and Pivot

```
D* ← CuiHeuristic(G)
function BranchP(S = (U_S, V_S), C = (U_C, V_C))
    if C = ∅ then
        if |E(S)| > |E(D*)| and |U_S| ≥ θ and |V_S| ≥ θ then
            D* ← S
            return
    C_0 ← {v ∈ C | d̄_S(v) = 0}
    if C_0 = ∅ or |C \ C_0| ≥ k − |Ē(S)| then
        u ← argmax_{v∈C} d̄_S
        (S', C') ← Update(S, C, u)
        BranchP(S' ∪ {u}, C')
        BranchP(S, C \ {u})
    else
        u ← argmin_{v∈C_0} d̄_C
        if d̄_C(u) > k − |Ē(S)| > 0 then
            (S', C') ← Update(S, C, u)
            BranchP(S' ∪ {u}, C')
            BranchP(S, C \ {u})
        else
            for v ∈ {u} ∪ N_C(u) do
                (S', C') ← Update(S, C, v)
                BranchP(S' ∪ {v}, C')
                C ← C \ {v}
```

Where D^* is the k-MDB of the graph.

The paper proposes several novel optimization techniques as well. These techniques are beyond the scope of this paper but were implemented during testing. These optimizations include several graph reduction techniques that remove nodes provably not part of a k-MDB.

4 Results

We use several graphs from `konect.cc`, a registry of bipartite graphs. We first applied the reductions proposed by [Cui et al.], and then ran both ACO and the [Cui et al.] heuristic. We ran ACO for a total of 5 iterations, and measured both the time to completion as well as the iterations and time to the best returned solution. We measured time and performance for the Cui Heuristic. We summarize our results below.

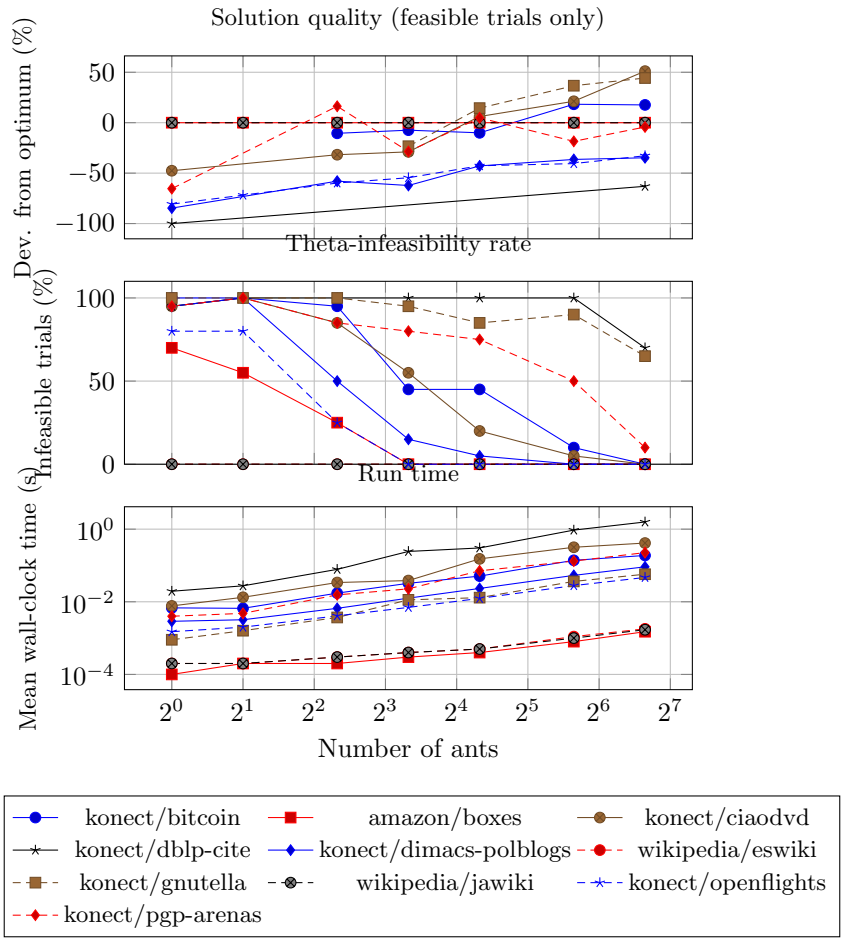

Table 1: $k = 2, \theta = 5$

Dataset	$\lvert U_G\rvert$	$\lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert$	$\lvert V_R\rvert$	ACO				Cui	
						$\lvert E(D^*)\rvert$	Time	ITB	TTB	$\lvert E(D^*)\rvert$	Time
Boxes	15237	1128	16239	14	14	23	0.0023	1	0.0008	23	0.0002
Music	100952	70519	130457	63	81	18	0.0088	1	0.0033	13	0.0003

5 Discussion

6 Conclusion

7 Works Cited

[1] D. Cui, R. Li, Q. Dai, H. Qin, and G. Wang, "On the Efficient Discovery of Maximum K-Defective Biclique," arXiv.org. Accessed: Aug. 11, 2026. [Online]. Available: https://arxiv.org/abs/2506.16121

[2] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant System: Optimization by a Colony of Cooperating Agents," IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics), vol. 26, no. 1, pp. 29–41, 1996, doi: 10.1109/3477.484436.

[3] S. Shah, "GCLIQUE: An Open Source Genetic Algorithm for the Maximum Clique Problem," Aug. 2020, doi: 10.36227/techrxiv.12814217.v1.